

TÌM HIỂU VỀ NODEJS VÀ XÂY DỰNG WEBSITE NHÀ HÀNG ẨM THỰC PHƯƠNG NAM TRÀ VINH

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

This image shows a single page of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

Trà Vinh, ngày tháng năm

Giáo viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Trà Vinh, ngày tháng năm

Giáo viên hướng dẫn

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy đã hướng dẫn tôi trong quá trình viết bài báo cáo Đồ án kết thúc môn. Trong thời gian qua, chúng tôi đã học được nhiều kiến thức mới trong lĩnh vực của ngành Công nghệ thông tin, trong suốt quá trình nghiên cứu cho đến kết thúc đồ án. Thông qua bài báo cáo Đồ án môn học này, tôi xin gửi lời cảm ơn đến thầy Nguyễn Bảo Ân giảng viên Khoa Công nghệ thông tin, đã đem lại cho chúng tôi những kiến thức hữu ích thông qua bài báo cáo Đồ án môn học với chủ đề **“TÌM HIỂU VỀ NODEJS VÀ XÂY DỰNG WEBSITE NHÀ HÀNG ẨM THỰC PHƯƠNG NAM TRÀ VINH”**.

Chúng tôi cũng gửi lời cảm ơn đến Trường Đại học Trà Vinh, Trường Kỹ thuật và Công nghệ, Khoa Công nghệ thông tin đã tạo đủ điều kiện cũng như các kiến thức tài liệu học tập, tài liệu tham khảo để tôi hoàn thành Đồ án. Trong quá trình viết báo cáo, tôi còn nhiều sai sót và khuyết điểm trong quá trình tìm hiểu và viết báo cáo. Chúng tôi rất mong nhận được sự đánh giá đóng góp ý kiến của Thầy, Cô để chúng tôi khắc phục và hoàn thiện hơn.

Chúng tôi xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	3
MỤC LỤC	4
DANH MỤC TỪ VIẾT TẮT	7
TÓM TẮT ĐỒ ÁN MÔN HỌC	9
CHƯƠNG 1. MỞ ĐẦU	10
1.1. GIỚI THIỆU	10
1.1.1. Bối cảnh và Tên dự án	10
1.1.2. Mục tiêu của dự án (Nghệ nghiệp vụ và Kỹ thuật)	10
1.1.3. Lý do chọn đề tài và phạm vi dự án	11
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU	13
2.1. Giới thiệu về Phân tích yêu cầu	13
2.2. Xác định các bên liên quan và người dùng mục tiêu (Stakeholders & Target Users)	13
2.2.1. Đặc tả nhu cầu ứng dụng phù hợp	14
2.3. Tính cần thiết của phần mềm phân tán, microservices và cloud	16
2.4. Yêu cầu chức năng (Functional Requirements)	17
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	20
3.1. Giới thiệu và Nguyên tắc thiết kế	20
3.2. Mô tả chức năng từng phần trong kiến trúc	21
3.3. Kiến trúc phần mềm dựa trên đám mây và định hướng Microservices	22
3.4. Thiết kế Kiến trúc tổng thể (System Architecture)	23
3.5. Sơ đồ kiến trúc phân lớp (Layered Architecture) của Backend:	24
3.6. Thiết kế Cơ sở dữ liệu (Database Design)	25

3.7. Thiết kế Giao diện Lập trình Ứng dụng (API Design)	29
3.8. Thiết kế giao diện (UI/UX): Ảnh chụp các màn hình chính và liên kết với bản thiết kế Figma	32
CHƯƠNG 4. Triển khai và công nghệ sử dụng	33
4.1. Công nghệ sử dụng	33
4.2. Quy trình CI/CD với GitHub Actions	34
4.3. Cấu hình Docker và quy trình triển khai	34
4.4. Kết quả kiểm thử docker	35
CHƯƠNG 5. Quản lý dự án	36
5.1. Vận dụng mô hình phát triển phần mềm Agile	36
CHƯƠNG 6. Kiểm thử	38
6.1. Chiến lược kiểm thử	38
6.2. Kết quả kiểm thử API	39
CHƯƠNG 7. ĐÁNH GIÁ VÀ KẾT LUẬN	40
7.1. Những khó khăn gặp phải	40
7.2. Bài học rút ra	40
7.3. Đề xuất cải tiến trong tương lai	40
CHƯƠNG 8. PHỤ LỤC	41
8.1. Hướng dẫn cài đặt và chạy ứng dụng	41
DANH MỤC TÀI LIỆU THAM KHẢO	42

DANH MỤC HÌNH ẢNH

Hình 3.1 : Sơ đồ kiến trúc phân lớp	24
Hình 3.2 : Mô hình dữ liệu quan hệ ERD	25
Hình 3.3: Giao diện bản vẽ FigMa	32
Hình 4.2: Kết quả kiểm thử Docker (Frontend, Backend, DataBase)	35
Hình 6.1: Kiểm thử các API	39

DANH MỤC TỪ VIẾT TẮT

STT	Ký hiệu chữ viết tắt	Chữ viết đầy đủ
1	API	Application Programming Interface
2	F&B	Food & Beverage
3	I/O	Input/Output
4	NPM	Node Package Manager
5	REST	Representational State Transfer
6	CRUD	Create, Read, Update, Delete
7	CI/CD	Continuous Integration/Continuous Deployment
8	CSDL	Cơ Sở Dữ Liệu
9	HTML	HyperText Markup Language
10	CSS	Cascading Style Sheets
11	SPA	Single Page Application
12	ACID	Atomicity, Consistency, Isolation, Durability
13	HTTP	HyperText Transfer Protocol
14	SQL	Structured Query Language
15	ERD	Entity-Relationship Diagram
16	PK	Primary Key (Khóa chính)

**TÌM HIỂU VỀ NODEJS VÀ XÂY DỰNG WEBSITE NHÀ HÀNG ẨM THỰC PHƯƠNG
NAM TRÀ VINH**

17	UI/UX	User Interface/User Experience
-----------	-------	--------------------------------

TÓM TẮT ĐỒ ÁN MÔN HỌC

Đồ án môn học với đề tài “Tìm hiểu về Node.js và xây dựng website nhà hàng Âm Thực Phương Nam Trà Vinh” được thực hiện với mục tiêu nghiên cứu nền tảng lập trình backend hiện đại Node.js, đồng thời áp dụng vào việc xây dựng một hệ thống website giới thiệu và quản lý hoạt động của nhà hàng. Trong quá trình thực hiện, nhóm đã tìm hiểu các khái niệm cơ bản về Node.js, Express.js, cũng như cách kết nối cơ sở dữ liệu MySQL để xây dựng một hệ thống có khả năng xử lý yêu cầu nhanh, gọn, hiệu quả. Website gồm các chức năng chính như: hiển thị thực đơn món ăn theo từng loại, đặt bàn trực tuyến, đặt món ăn, quản lý hóa đơn và chi tiết hóa đơn, quản lý danh mục món ăn từ giao diện admin. Giao diện được thiết kế thân thiện, dễ sử dụng, tương thích nhiều thiết bị, sử dụng HTML, CSS, (hoặc Tailwind CSS). Kết quả đạt được là một website hoàn chỉnh, thể hiện sự kết hợp giữa kiến thức lý thuyết và thực tiễn, giúp sinh viên rèn luyện kỹ năng lập trình web backend cũng như phát triển tư duy thiết kế hệ thống. Đây là một bước đệm quan trọng để hướng đến các dự án lớn hơn trong tương lai.

CHƯƠNG 1. MỞ ĐẦU

1.1. GIỚI THIỆU

1.1.1. Bối cảnh và Tên dự án

Tên dự án: Website Nhà hàng Âm thực Phương Nam Trà Vinh.

Bối cảnh: Trong bối cảnh cuộc cách mạng công nghiệp 4.0, chuyển đổi số là xu hướng tất yếu của mọi ngành nghề, bao gồm cả lĩnh vực ẩm thực và nhà hàng (F&B). Một nhà hàng, dù có món ăn ngon và không gian đẹp đến đâu, cũng sẽ gặp khó khăn trong việc tiếp cận khách hàng nếu không có sự hiện diện chuyên nghiệp trên không gian mạng. Đặc biệt tại các địa phương như Trà Vinh, việc sở hữu một website không chỉ giúp quảng bá thương hiệu mà còn là một công cụ vận hành hiệu quả, giúp nhà hàng tối ưu hóa quy trình đặt bàn và quản lý. Dự án này được thực hiện nhằm giải quyết bài toán đó cho nhà hàng "Âm thực Phương Nam", mang đến một giải pháp số hóa toàn diện.

1.1.2. Mục tiêu của dự án (Nghệ thuật và Kỹ thuật)

Dự án được xây dựng với hai nhóm mục tiêu song song:

Mục tiêu nghiệp vụ:

Nâng cao hình ảnh thương hiệu: Xây dựng một website chuyên nghiệp, hiện đại, phản ánh đúng tinh thần và chất lượng của nhà hàng.

Tăng hiệu quả kinh doanh: Cung cấp kênh đặt bàn trực tuyến, giảm tải cho nhân viên và tăng tỷ lệ lấp đầy bàn, đặc biệt vào các giờ cao điểm.

Tối ưu hóa vận hành: Cung cấp cho quản lý nhà hàng một hệ thống quản trị (admin panel) trực quan để dễ dàng cập nhật thực đơn, quản lý các lượt đặt bàn.

Mở rộng tệp khách hàng: Tiếp cận các khách hàng tiềm năng, khách du lịch thông qua công cụ tìm kiếm và các kênh trực tuyến.

Mục tiêu kỹ thuật:

Nghiên cứu và làm chủ Node.js: Tìm hiểu sâu về môi trường runtime Node.js, mô hình bất đồng bộ, non-blocking I/O và hệ sinh thái NPM.

Xây dựng RESTful API: Thiết kế và triển khai một hệ thống API theo chuẩn REST, đảm bảo tính nhất quán, khả năng mở rộng và dễ dàng tích hợp.

Áp dụng các công nghệ hiện đại: Thực hành và vận dụng các công nghệ phổ biến như Express.js, MySQL, Tailwind CSS, Docker và GitHub Actions vào một sản phẩm thực tế.

Hoàn thiện quy trình phát triển phần mềm: Áp dụng một quy trình làm việc chuyên nghiệp từ khâu phân tích yêu cầu, thiết kế, lập trình, kiểm thử đến triển khai.

1.1.3. Lý do chọn đề tài và phạm vi dự án

Lý do chọn đề tài: Đề tài này hội tụ đủ các yếu tố của một đề án tốt: tính thực tiễn cao, phạm vi vừa phải và là cơ hội để ứng dụng các công nghệ đang là xu hướng. Việc xây dựng một website cho nhà hàng đòi hỏi sự kết hợp nhuần nhuyễn giữa kỹ năng backend (xử lý logic, CSDL), frontend (xây dựng giao diện), và DevOps (triển khai, tự động hóa), mang lại một cái nhìn toàn diện về phát triển sản phẩm web.

Phạm vi dự án:

Trong phạm vi (In-scope):

- Các chức năng cho người dùng: xem thông tin nhà hàng, xem thực đơn, đặt bàn.
- Các chức năng cho quản trị viên: đăng nhập, quản lý thực đơn (CRUD), quản lý danh mục, quản lý đặt bàn.
- Triển khai ứng dụng trên một máy chủ ảo sử dụng Docker.
- Thiết lập CI/CD cơ bản với GitHub Actions.
- Ngoài phạm vi (Out-of-scope) cho phiên bản này:

TÌM HIỂU VỀ NODEJS VÀ XÂY DỰNG WEBSITE NHÀ HÀNG ẨM THỰC PHƯƠNG NAM TRÀ VINH

- Hệ thống tài khoản cho khách hàng.
- Chức năng đặt món và giao hàng trực tuyến.
- Tích hợp cổng thanh toán online.
- Chức năng đánh giá, bình luận của khách hàng.

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU

2.1. Giới thiệu về Phân tích yêu cầu

Phân tích yêu cầu là giai đoạn nền tảng và quan trọng nhất trong mọi quy trình phát triển phần mềm. Mục tiêu của giai đoạn này là xác định, ghi nhận, và xác thực các nhu cầu và mong đợi của các bên liên quan đối với sản phẩm. Việc phân tích yêu cầu một cách kỹ lưỡng giúp đảm bảo rằng sản phẩm cuối cùng sẽ giải quyết đúng vấn đề nghiệp vụ, đáp ứng được mục tiêu đề ra, và giảm thiểu rủi ro thay đổi tốn kém ở các giai đoạn sau.

Trong khuôn khổ dự án "Website Nhà hàng Âm thực Phương Nam Trà Vinh", chương này sẽ tập trung vào việc làm rõ các yêu cầu chức năng (hệ thống phải làm gì) và các yêu cầu phi chức năng (hệ thống phải như thế nào), tạo cơ sở vững chắc cho các giai đoạn thiết kế, triển khai và kiểm thử.

2.2. Xác định các bên liên quan và người dùng mục tiêu (Stakeholders & Target Users)

Để xây dựng một sản phẩm hiệu quả, việc đầu tiên là phải hiểu rõ đối tượng sử dụng và các bên có quyền lợi liên quan đến hệ thống.

Khách hàng (Customer/Guest):

Đặc điểm: Là đối tượng người dùng chính của trang web công khai. Họ có thể là khách hàng quen, khách hàng mới, hoặc khách du lịch đang tìm kiếm một địa điểm ăn uống tại Trà Vinh. Độ tuổi và mức độ thành thạo công nghệ đa dạng.

Mục tiêu:

- Tìm kiếm thông tin về nhà hàng một cách nhanh chóng (địa chỉ, giờ mở cửa, không gian).
- Khám phá thực đơn và các món ăn đặc sắc trước khi đến.
- Đặt bàn trực tuyến một cách tiện lợi, tránh phải gọi điện hoặc chờ đợi.
- Biết được các chương trình khuyến mãi đang diễn ra

Quản lý/Nhân viên nhà hàng (Admin/Staff):

Đặc điểm: Là người dùng của hệ thống quản trị (Admin Panel). Họ chịu trách nhiệm vận hành các hoạt động hàng ngày của nhà hàng liên quan đến website.

Mục tiêu:

- Có một công cụ tập trung để quản lý thông tin hiển thị trên website.
- Dễ dàng cập nhật thực đơn (thêm món, sửa giá, thay đổi hình ảnh) mà không cần kiến thức kỹ thuật.
- Tiếp nhận và xử lý các yêu cầu đặt bàn từ khách hàng một cách có hệ thống.
- Theo dõi và xác nhận các lượt đặt bàn để sắp xếp chỗ ngồi hợp lý.

2.2.1. Đặc tả nhu cầu ứng dụng phù hợp

Để đảm bảo hệ thống được xây dựng phù hợp với nhu cầu thực tế, nhóm đã phân tích hành vi và kỳ vọng của người dùng thông qua các tình huống sử dụng (user stories) sau:

Đặt bàn nhanh chóng, không cần gọi điện

User Story:

Là một khách hàng đang có nhu cầu dùng bữa tại nhà hàng, tôi muốn đặt bàn online trong vài bước đơn giản để không phải gọi điện và chờ xác nhận thủ công.

Giải pháp chức năng:

→ Hệ thống cung cấp một form đặt bàn trực tuyến với các trường: họ tên, số điện thoại, ngày giờ, số lượng khách và xác nhận tự động.

Khám phá thực đơn dễ dàng, hấp dẫn

User Story:

Là một khách lần đầu truy cập website, tôi muốn xem thực đơn được phân loại rõ ràng kèm hình ảnh minh họa để dễ chọn món hơn.

Giải pháp chức năng:

→ Giao diện thực đơn trực quan, có bộ lọc theo danh mục (khai vị, món chính...), thanh tìm kiếm và mô tả chi tiết từng món ăn.

Quản lý nhà hàng tiện lợi, không cần kỹ thuật

User Story:

Là quản lý nhà hàng, tôi muốn cập nhật món ăn, danh mục, đơn đặt bàn một cách nhanh chóng mà không cần hiểu lập trình.

Giải pháp chức năng:

→ Tích hợp Admin Panel với giao diện CRUD thân thiện để thêm/sửa/xóa món ăn, danh mục, xem đơn đặt bàn và thay đổi trạng thái.

Tăng sự tin cậy thông qua trải nghiệm mượt mà

User Story:

Là khách hàng, tôi muốn website phản hồi nhanh, bảo mật và có giao diện đẹp để tôi cảm thấy yên tâm khi sử dụng dịch vụ.

Giải pháp phi chức năng:

→ Hệ thống đảm bảo hiệu suất cao, giao diện responsive, sử dụng HTTPS, JWT cho bảo mật, và triển khai trên nền Docker kết hợp CI/CD để duy trì ổn định.

2.3. Tính cần thiết của phần mềm phân tán, microservices và cloud

Trong bối cảnh công nghệ hiện đại, việc phát triển phần mềm theo hướng phân tán và triển khai trên nền tảng đám mây (cloud) không còn là lựa chọn, mà là xu hướng tất yếu. Đối với dự án website nhà hàng “Ẩm thực Phương Nam Trà Vinh”, tính cần thiết của việc áp dụng mô hình này thể hiện rõ ở các khía cạnh sau:

Khả năng mở rộng (Scalability): Khi lưu lượng truy cập tăng cao vào giờ cao điểm hoặc dịp lễ, hệ thống cần khả năng mở rộng linh hoạt mà không ảnh hưởng đến hiệu năng. Cloud infrastructure như Docker và CI/CD giúp scale nhanh gọn.

Tính sẵn sàng cao (High Availability): Với phần mềm phân tán, việc phân chia thành các thành phần nhỏ (service) và triển khai trên cloud giúp đảm bảo hệ thống luôn hoạt động ổn định, ngay cả khi một thành phần bị lỗi.

Tách biệt chức năng – Dễ bảo trì (Modularity & Maintainability): Kiến trúc microservices cho phép chia hệ thống thành các module độc lập như: quản lý đặt bàn, thực đơn, hóa đơn... giúp dễ nâng cấp, triển khai từng phần mà không ảnh hưởng toàn cục.

Triển khai nhanh – DevOps hiện đại: Sử dụng Docker và GitHub Actions hỗ trợ quy trình CI/CD tự động, giúp nhóm triển khai ứng dụng nhanh chóng, liên tục cập nhật và đảm bảo chất lượng.

Phù hợp với định hướng số hóa của ngành F&B: Nhà hàng hiện đại không chỉ cần một website, mà còn là một nền tảng số linh hoạt, sẵn sàng tích hợp với AI, chatbot, mobile app hoặc hệ thống giao hàng trong tương lai. Kiến trúc microservices và cloud chính là bước đệm cần thiết.

2.4. Yêu cầu chức năng (Functional Requirements)

Dựa trên việc phân tích các bên liên quan, các yêu cầu chức năng của hệ thống được chia thành hai phân hệ chính.

Trang chủ

- Hiện thị một banner lớn, bắt mắt với hình ảnh/video về không gian hoặc món ăn đặc trưng của nhà hàng.
- Cung cấp một đoạn giới thiệu ngắn gọn về câu chuyện và phong cách của "Ẩm thực Phương Nam".
- Trưng bày một số món ăn nổi bật (Best Seller/Signature Dishes) để thu hút sự chú ý.
- Hiện thị các chương trình khuyến mãi hoặc sự kiện đặc biệt (nếu có).
- Cung cấp các nút kêu gọi hành động (Call-to-Action) rõ ràng như "Đặt bàn ngay" và "Xem thực đơn".

Xem thực đơn

- Hiện thị danh sách các món ăn được phân loại rõ ràng theo các danh mục (ví dụ: Món khai vị, Món chính, Lẩu, Tráng miệng, Đồ uống).
- Mỗi món ăn phải hiển thị đầy đủ thông tin: Tên món, Hình ảnh minh họa, Mô tả ngắn gọn, Giá bán.
- Cung cấp chức năng lọc món ăn theo danh mục.
- Cung cấp thanh tìm kiếm cho phép người dùng tìm món ăn theo tên.
- Đặt bàn trực tuyến
- Cung cấp một form đặt bàn trực quan, dễ sử dụng.
- Các trường thông tin bắt buộc bao gồm: Họ và tên, Số điện thoại, Số lượng khách, Ngày đặt, Giờ đặt.

Hệ thống phải thực hiện kiểm tra dữ liệu đầu vào:

- Số điện thoại phải đúng định dạng.
- Ngày và giờ đặt phải trong tương lai.
- Số lượng khách phải là số dương.
- Sau khi gửi yêu cầu thành công, hệ thống phải hiển thị một thông báo xác nhận cho người dùng, cho biết yêu cầu đã được ghi nhận và đang chờ xử lý.

Trang liên hệ

- Hiển thị đầy đủ thông tin liên lạc của nhà hàng: Địa chỉ, Số điện thoại, Email.
- Hiển thị giờ mở cửa, đóng cửa các ngày trong tuần.
- Tích hợp một bản đồ Google Maps tương tác để người dùng dễ dàng tìm đường đi.

Quản lý Thực đơn (CRUD)

- Create: Cho phép quản trị viên thêm một món ăn mới với các thông tin: tên, mô tả, giá, danh mục, và tải lên hình ảnh.
- Read: Hiển thị danh sách tất cả món ăn trong hệ thống dưới dạng bảng, có hỗ trợ tìm kiếm, lọc và phân trang.
- Update: Cho phép quản trị viên chỉnh sửa thông tin của một món ăn đã tồn tại.
- Delete: Cho phép quản trị viên xóa một món ăn khỏi thực đơn.
- FC-06: Quản lý Danh mục (CRUD)
- Cung cấp giao diện để quản trị viên có thể thêm, sửa, xóa các danh mục món ăn. Việc này giúp tổ chức thực đơn một cách linh hoạt.

Quản lý Đặt bàn

- Hiện thị danh sách tất cả các yêu cầu đặt bàn từ khách hàng, sắp xếp theo thời gian gần nhất.
- Mỗi yêu cầu đặt bàn hiển thị đầy đủ thông tin khách hàng và trạng thái hiện tại (Mới, Đã xác nhận, Đã hủy).
- Cho phép quản trị viên thay đổi trạng thái của một yêu cầu đặt bàn. Ví dụ, chuyển từ "Mới" sang "Đã xác nhận".
- Cung cấp chức năng lọc các yêu cầu đặt bàn theo ngày hoặc theo trạng thái.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Giới thiệu và Nguyên tắc thiết kế

Giai đoạn thiết kế hệ thống là quá trình kiến tạo "bản vẽ chi tiết" cho ngôi nhà phần mềm, dựa trên "bản phác thảo yêu cầu" đã được định hình ở chương trước. Đây là giai đoạn mang tính quyết định, ảnh hưởng trực tiếp đến chất lượng, hiệu suất, và khả năng phát triển của sản phẩm trong tương lai. Mục tiêu của chương này là diễn giải các yêu cầu nghiệp vụ thành một cấu trúc kỹ thuật logic, rõ ràng và khả thi.

Để đảm bảo chất lượng của bản thiết kế, chúng tôi tuân thủ các nguyên tắc thiết kế phần mềm cốt lõi sau:

Phân chia Trách nhiệm (Separation of Concerns - SoC): Tách biệt hệ thống thành các thành phần riêng biệt, mỗi thành phần có một trách nhiệm duy nhất và rõ ràng (ví dụ: tầng giao diện, tầng nghiệp vụ, tầng dữ liệu). Điều này giúp mã nguồn dễ hiểu, dễ bảo trì và dễ kiểm thử hơn.

Tính Module hóa (Modularity): Xây dựng hệ thống từ các module độc lập, có khả năng tái sử dụng và kết nối với nhau thông qua các giao diện (API) được định nghĩa rõ ràng.

Tính Đơn giản (Keep It Simple, Stupid - KISS): Ưu tiên các giải pháp đơn giản và hiệu quả nhất, tránh các thiết kế phức tạp không cần thiết có thể gây khó khăn cho việc bảo trì và mở rộng sau này.

Không lặp lại chính mình (Don't Repeat Yourself - DRY): Tránh lặp lại mã nguồn hoặc logic ở nhiều nơi khác nhau bằng cách trừu tượng hóa và tái sử dụng.

3.2. Mô tả chức năng từng phần trong kiến trúc

Hệ thống website "Ẩm Thực Phương Nam" được xây dựng theo mô hình phân lớp, bao gồm ba lớp chính: Frontend (Client), Backend (API Server) và Cơ sở dữ liệu (Database). Bên cạnh đó, hệ thống còn tích hợp lớp triển khai (Deployment layer) sử dụng các công nghệ DevOps hiện đại.

Các chức năng cụ thể của từng thành phần được mô tả như sau:

Thành phần	Chức năng chính
Frontend (Giao diện)	- Hiển thị giao diện trang chủ, thực đơn, thông tin nhà hàng, form đặt bàn. - Nhận thông tin đầu vào từ người dùng và gửi đến backend qua các API.
API Server (Backend)	- Xử lý nghiệp vụ chính: quản lý món ăn, đặt bàn, khách hàng, hóa đơn. - Xây dựng bằng Node.js và Express theo kiến trúc RESTful API.
Database (MySQL)	- Lưu trữ dữ liệu có cấu trúc: danh sách món ăn, thông tin khách hàng, đơn đặt bàn, chi tiết hóa đơn... - Đảm bảo toàn vẹn và bảo mật dữ liệu.
Triển khai (DevOps)	- Sử dụng Docker để đóng gói toàn bộ hệ thống thành container. - Tự động hóa build và triển khai qua GitHub Actions (CI/CD). - Sẵn sàng triển khai lên các nền tảng cloud.

3.3. Kiến trúc phần mềm dựa trên đám mây và định hướng

Microservices

Hiện tại, nhóm lựa chọn triển khai hệ thống theo kiến trúc Monolithic, tức toàn bộ backend và API được tổ chức trong cùng một ứng dụng Node.js duy nhất. Mô hình này phù hợp với nhóm sinh viên có quy mô nhỏ, giúp quá trình phát triển và triển khai diễn ra nhanh chóng, dễ quản lý.

Tuy nhiên, ngay từ đầu nhóm đã hướng tới việc thiết kế hệ thống theo mô hình module hóa – các chức năng như quản lý món ăn, quản lý khách hàng, quản lý đặt bàn được tổ chức tách biệt để có thể dễ dàng chuyển đổi sang kiến trúc Microservices trong tương lai.

3.4. Thiết kế Kiến trúc tổng thể (System Architecture)

Dự án áp dụng mô hình kiến trúc Client-Server hiện đại, bao gồm các thành phần chính sau:

Frontend (Client-Side): Một Single Page Application (SPA).

Lý do lựa chọn: SPA mang lại trải nghiệm người dùng mượt mà, nhanh chóng và tương tác cao, gần giống với ứng dụng di động. Sau lần tải đầu tiên, việc điều hướng giữa các trang diễn ra gần như tức thì vì chỉ có dữ liệu cần thiết được tải về từ server qua API, thay vì tải lại toàn bộ trang HTML. Điều này đặc biệt hiệu quả cho các trang có tính tương tác cao như xem thực đơn, tạo đơn hàng.

Backend (Server-Side): Một API Server theo kiến trúc Monolithic.

Lý do lựa chọn: Với quy mô hiện tại của dự án, kiến trúc Monolithic (toàn bộ logic backend nằm trong một khối ứng dụng duy nhất) là lựa chọn hợp lý. Nó giúp đơn giản hóa quá trình phát triển, kiểm thử và triển khai ban đầu. Việc quản lý một codebase và một quy trình triển khai duy nhất hiệu quả hơn so với việc phải quản lý nhiều dịch vụ nhỏ lẻ (microservices). Kiến trúc này có thể được tái cấu trúc thành microservices trong tương lai nếu ứng dụng phát triển lớn mạnh và yêu cầu khả năng mở rộng ở từng chức năng cụ thể.

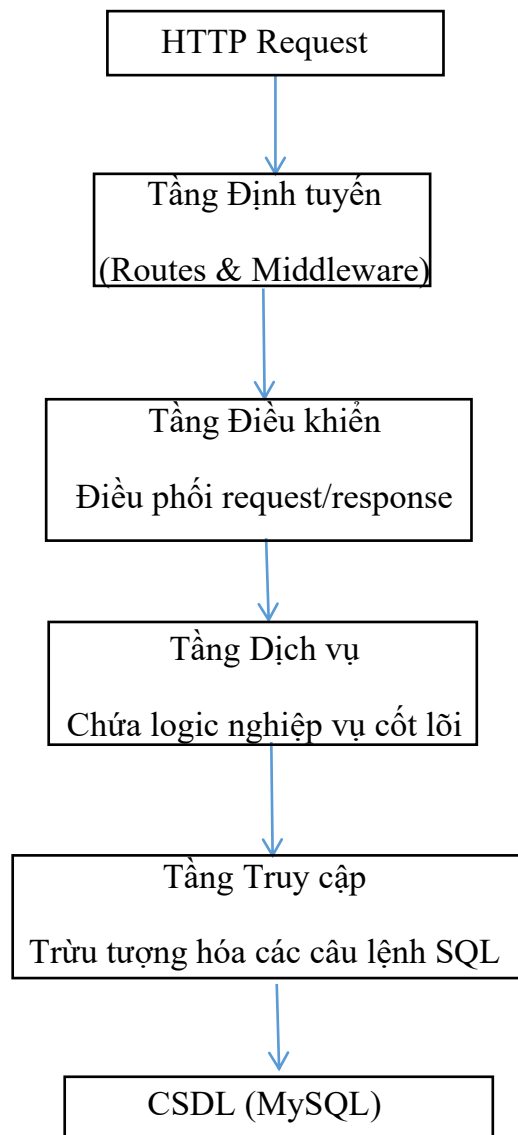
Database: Hệ quản trị cơ sở dữ liệu quan hệ MySQL.

Lý do lựa chọn: Dữ liệu của nhà hàng có tính cấu trúc và quan hệ cao (khách hàng có nhiều hóa đơn, hóa đơn có nhiều chi tiết, món ăn thuộc về một loại...). MySQL cung cấp các ràng buộc toàn vẹn dữ liệu mạnh mẽ (khóa chính, khóa ngoại) và tuân thủ thuộc tính ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo các giao dịch quan trọng như tạo đơn hàng được thực hiện một cách an toàn và nhất quán.

Dịch vụ bên ngoài (External Service): Google Gemini AI API.

Lý do lựa chọn: Tích hợp AI giúp tạo ra điểm nhấn khác biệt và gia tăng giá trị cho người dùng. Thay vì tự xây dựng một mô hình phức tạp, việc sử dụng API của một dịch vụ AI hàng đầu như Gemini cho phép chúng ta nhanh chóng tích hợp các tính năng thông minh với chi phí và thời gian phát triển tối thiểu.

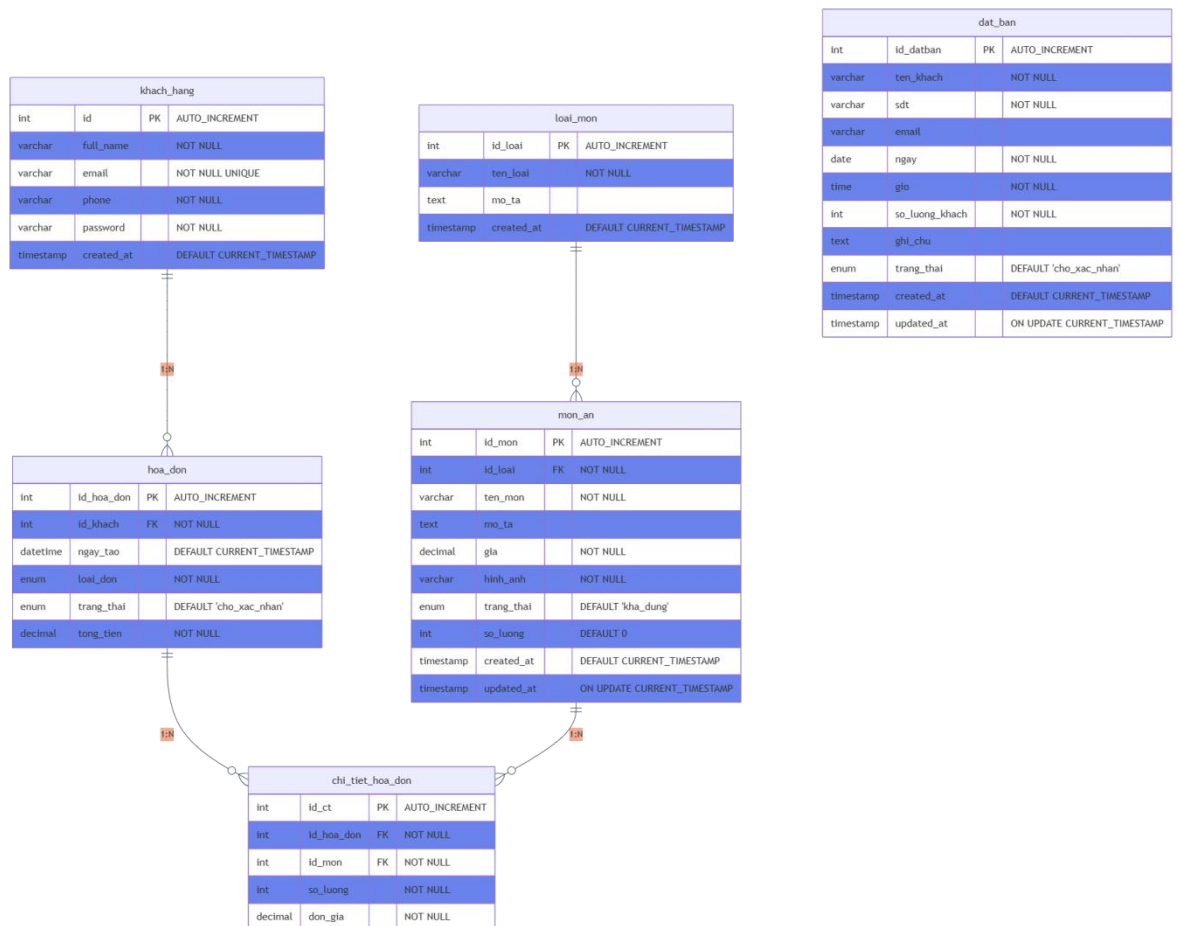
3.5. Sơ đồ kiến trúc phân lớp (Layered Architecture) của Backend:



Hình 3.1 : Sơ đồ kiến trúc phân lớp

3.6. Thiết kế Cơ sở dữ liệu (Database Design)

(Sơ đồ ERD như phiên bản trước, thể hiện rõ các mối quan hệ 1-Nhiều và Nhiều-Nhiều)



Hình 3.2 : Mô hình dữ liệu quan hệ ERD

Mô hình bao gồm 6 thực thể (bảng) chính, mỗi thực thể đại diện cho một đối tượng nghiệp vụ cụ thể trong hệ thống nhà hàng. Các thực thể này được liên kết với nhau thông qua các mối quan hệ được định nghĩa bằng khóa chính (PK) và khóa ngoại (FK).

1. Thực thể `loai_mon` (Loại Món)

Mục đích: Bảng này đóng vai trò là dữ liệu chủ (master data), dùng để phân loại các món ăn trong thực đơn. Việc phân loại giúp cho việc quản lý thực đơn trở nên khoa học và giúp khách hàng dễ dàng tìm kiếm (ví dụ: Món Khai Vị, Món Chính, Lẩu, Tráng Miệng...).

Các thuộc tính chính:

`id_loai` (PK): Khóa chính, định danh duy nhất cho mỗi loại món.

`ten_loai`: Tên của danh mục (ví dụ: "Món Chính").

Mối quan hệ: Có mối quan hệ Một-Nhiều (1:N) với thực thể `mon_an`. Một loại món có thể bao gồm nhiều món ăn khác nhau.

2. Thực thể `khach_hang` (Khách Hàng)

Mục đích: Lưu trữ thông tin về các khách hàng đã đăng ký tài khoản. Bảng này là nền tảng cho các chức năng nâng cao như quản lý khách hàng thân thiết, xem lại lịch sử đặt hàng, và cá nhân hóa trải nghiệm.

Các thuộc tính chính:

`id` (PK): Khóa chính, định danh duy nhất cho mỗi khách hàng.

`full_name`, `email`, `phone`: Thông tin cá nhân của khách hàng.

`password`: Lưu mật khẩu đã được mã hóa của khách hàng để xác thực đăng nhập.

Mối quan hệ: Có mối quan hệ Một-Nhiều (1:N) với thực thể `hoa_don`. Một khách hàng có thể có nhiều hóa đơn (đơn hàng) khác nhau.

3. Thực thể mon_an (Món Ăn)

Mục đích: Đây là thực thể trung tâm, chứa thông tin chi tiết về từng món ăn trong thực đơn của nhà hàng.

Các thuộc tính chính:

id_mon (PK): Khóa chính, định danh duy nhất cho mỗi món ăn.

id_loai (FK): Khóa ngoại, liên kết đến bảng loai_mon để xác định món ăn này thuộc danh mục nào.

ten_mon, mo_ta, gia, hinh_anh: Các thông tin hiển thị cho khách hàng.

trang_thai, so_luong: Các thuộc tính quan trọng cho việc quản lý vận hành, cho biết món ăn còn hàng hay không và quản lý tồn kho.

Mối quan hệ:

Nhiều-Một (N:1) với loai_mon: Nhiều món ăn thuộc về một loại món.

Một-Nhiều (1:N) với chi_tiet_hoa_don: Một món ăn có thể xuất hiện trong nhiều chi tiết hóa đơn khác nhau (tức là được nhiều khách hàng khác nhau đặt mua).

4. Thực thể hoa_don (Hóa Đơn)

Mục đích: Đại diện cho một giao dịch, một đơn hàng cụ thể của khách hàng. Đây là một thực thể giao dịch (transactional entity) cốt lõi.

Các thuộc tính chính:

id_hoa_don (PK): Khóa chính, định danh duy nhất cho mỗi hóa đơn.

id_khach (FK): Khóa ngoại, liên kết đến khách hàng đã tạo hóa đơn này.

ngay_tao, loai_don (tại chỗ/giao hàng), trang_thai (đang xử lý/hoàn thành...): Ghi lại thông tin và trạng thái của đơn hàng.

tong_tien: Tổng giá trị của hóa đơn.

Mối quan hệ:

Nhiều-Một (N:1) với `khach_hang`: Nhiều hóa đơn có thể thuộc về cùng một khách hàng.

Một-Nhiều (1:N) với `chi_tiet_hoa_don`: Một hóa đơn bao gồm nhiều dòng chi tiết các món ăn.

5. Thực thể `chi_tiet_hoa_don` (Chi Tiết Hóa Đơn)

Mục đích: Đây là một bảng trung gian (junction table), có vai trò giải quyết mối quan hệ Nhiều-Nhiều (N:N) giữa `hoa_don` và `mon_an`. Một hóa đơn có thể có nhiều món ăn, và một món ăn có thể nằm trong nhiều hóa đơn.

Các thuộc tính chính:

`id_ct` (PK): Khóa chính của bảng.

`id_hoa_don` (FK): Liên kết đến hóa đơn mà chi tiết này thuộc về.

`id_mon` (FK): Liên kết đến món ăn được đặt trong chi tiết này.

`so_luong`: Số lượng của món ăn đó trong hóa đơn.

`don_gia`: Thuộc tính cực kỳ quan trọng. Việc lưu lại đơn giá tại thời điểm mua hàng đảm bảo rằng dù giá món ăn trong bảng `mon_an` có thay đổi trong tương lai, hóa đơn cũ vẫn phản ánh đúng giá trị giao dịch tại thời điểm đó.

Mối quan hệ: Kết nối `hoa_don` và `mon_an`, thể hiện chi tiết "ai đã mua cái gì, số lượng bao nhiêu, với giá nào".

6. Thực thể `dat_ban` (Đặt Bàn)

Mục đích: Lưu trữ các yêu cầu đặt bàn của khách hàng.

Các thuộc tính chính:

`id_datban` (PK): Khóa chính, định danh duy nhất cho mỗi lượt đặt bàn.

`ten_khach`, `sdt`, `email`, `ngay`, `gio`, `so_luong_khach`: Các thông tin cần thiết để ghi nhận một yêu cầu đặt bàn.

3.7. Thiết kế Giao diện Lập trình Ứng dụng (API Design)

Hãy tưởng tượng API (Application Programming Interface) giống như một người phục vụ trong nhà hàng.

Khách hàng (Frontend - Giao diện web): Muốn gọi món, xem thực đơn, hay thanh toán.

Nhà bếp (Backend - Máy chủ và Cơ sở dữ liệu): Nơi chế biến món ăn và xử lý mọi thứ.

Người phục vụ (API): Là người trung gian, nhận yêu cầu từ khách hàng, chuyển đến nhà bếp, và mang kết quả (món ăn) từ nhà bếp trả lại cho khách hàng.

Nếu không có người phục vụ (API), khách hàng sẽ không biết cách giao tiếp với nhà bếp. Vì vậy, việc thiết kế một "người phục vụ" chuyên nghiệp, hiểu rõ quy trình là rất quan trọng để hệ thống hoạt động trơn tru.

Để "người phục vụ" của chúng tôi làm việc hiệu quả, chúng tôi đặt ra một vài quy tắc đơn giản:

Tên gọi rõ ràng: Chúng tôi dùng các tên gọi dễ hiểu cho các yêu cầu. Ví dụ, để lấy danh sách các món ăn, chúng tôi dùng địa chỉ là `/api/mon-an`.

Hành động rõ ràng: Chúng tôi dùng các "hành động" tiêu chuẩn của Internet:

GET: Để xem thông tin (ví dụ: xem danh sách món ăn).

POST: Để tạo mới một thứ gì đó (ví dụ: tạo một đơn hàng mới).

PUT: Để cập nhật thông tin (ví dụ: sửa giá một món ăn).

DELETE: Để xóa một thứ gì đó (ví dụ: xóa một món ăn khỏi thực đơn).

Phản hồi rõ ràng: Khi yêu cầu được xử lý, "người phục vụ" sẽ phản hồi lại với các mã trạng thái rõ ràng:

Mã 200 (OK): Mọi thứ đều ổn, yêu cầu của bạn đã được thực hiện.

Mã 201 (Created): Bạn đã tạo mới thành công một thứ gì đó.

Mã 400 (Bad Request): Yêu cầu của bạn bị sai (ví dụ: điền thiếu thông tin).

Mã 404 (Not Found): Không tìm thấy thứ bạn yêu cầu.

Mã 500 (Server Error): Có lỗi xảy ra ở "nhà bếp", xin hãy thử lại sau.

Hệ thống của chúng tôi có các nhóm "người phục vụ" chuyên trách cho từng khu vực:

API Quản lý Món ăn (/api/mon-an, /api/loai-mon)

Chức năng: Giúp khách hàng xem thực đơn, xem chi tiết từng món. Giúp người quản lý thêm, sửa, xóa món ăn và danh mục.

Ví dụ: GET /api/mon-an sẽ trả về danh sách tất cả món ăn.

API Quản lý Khách hàng & Đăng nhập (/api/auth)

Chức năng: Giúp khách hàng đăng ký, đăng nhập vào tài khoản của mình.

Ví dụ: POST /api/auth/login sẽ kiểm tra tên đăng nhập và mật khẩu, nếu đúng sẽ cho phép khách hàng vào hệ thống.

API Quản lý Đặt bàn (/api/dat-ban)

Chức năng: Giúp khách hàng gửi yêu cầu đặt bàn. Giúp người quản lý xem và xác nhận các yêu cầu đó.

Ví dụ: POST /api/dat-ban sẽ lưu lại thông tin đặt bàn của khách.

API Quản lý Hóa đơn (/api/hoa-don)

Chức năng: Chức năng quan trọng nhất, giúp khách hàng tạo đơn hàng và thanh toán. Giúp người quản lý xem các đơn hàng đã được tạo.

Ví dụ: POST /api/hoa-don sẽ nhận thông tin giỏ hàng của khách và tạo ra một hóa đơn mới.

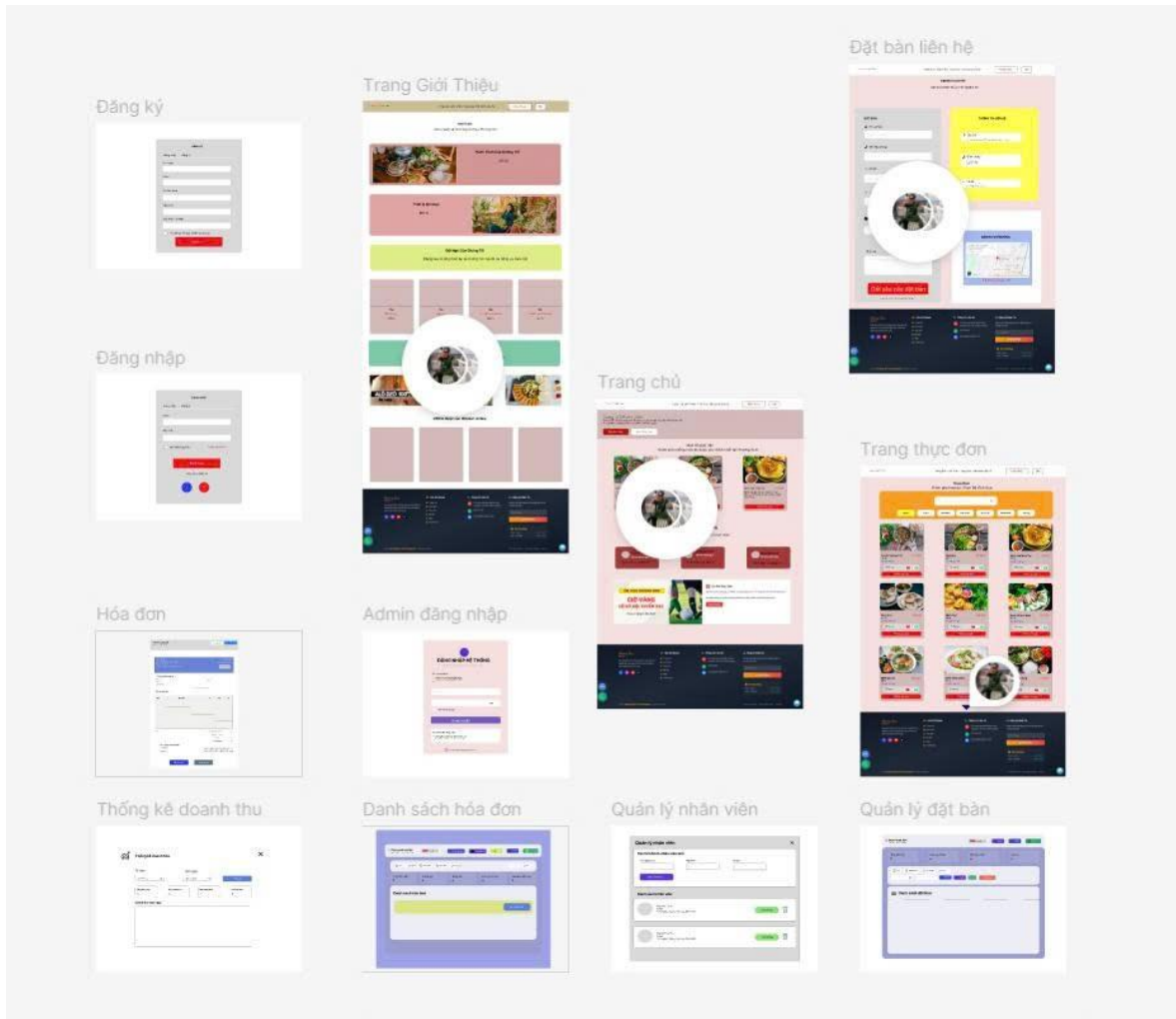
API Trí tuệ nhân tạo (/api/gemini)

Chức năng: Đây là "người phục vụ đặc biệt", có thể đưa ra gợi ý thông minh.

TÌM HIỂU VỀ NODEJS VÀ XÂY DỰNG WEBSITE NHÀ HÀNG ẨM THỰC PHƯƠNG NAM TRÀ VINH

Ví dụ: POST /api/gemini/suggest-meal - Khách hàng nói "tôi muốn một bữa ăn cho cặp đôi", API này sẽ hỏi AI và trả về một thực đơn gợi ý lãng mạn.

3.8. Thiết kế giao diện (UI/UX): Ảnh chụp các màn hình chính và liên kết với bản thiết kế Figma



Hình 3.3: Giao diện bản vẽ FigMa

CHƯƠNG 4. TRIỂN KHAI VÀ CÔNG NGHỆ SỬ DỤNG

4.1. Công nghệ sử dụng

Dự án “**ẨM THỰC PHƯƠNG NAM**” được xây dựng trên nền tảng công nghệ hiện đại, dễ triển khai và mở rộng. Nhóm lựa chọn các công nghệ phù hợp với mục tiêu thực tế, bao gồm:

Backend:

- Node.js – môi trường chạy JavaScript phía server.
- Express.js – framework nhẹ giúp tổ chức API rõ ràng, hỗ trợ routing và middleware.

Cơ sở dữ liệu:

- MySQL – hệ quản trị cơ sở dữ liệu quan hệ phổ biến, phù hợp xử lý thông tin dạng bảng như món ăn, khách hàng, hóa đơn.

Frontend:

- HTML5/CSS3 – ngôn ngữ cấu trúc và trình bày giao diện web.
- Tailwind CSS – thư viện tiện ích CSS giúp thiết kế nhanh, responsive, đẹp mắt.

Quản lý mã nguồn và CI/CD:

- GitHub – nền tảng quản lý phiên bản.
- GitHub Actions – công cụ tự động hóa kiểm thử và triển khai.

Triển khai & Ảo hóa:

- Docker – công cụ container hóa toàn bộ ứng dụng.
- Railway/Render/VPS – nền tảng cloud triển khai thực tế.

4.2. Quy trình CI/CD với GitHub Actions

Continuous Integration (CI) và Continuous Deployment (CD) là nền tảng của phát triển phần mềm hiện đại. Nhóm sử dụng GitHub Actions để:

Tự động hóa kiểm tra source code:

- Kiểm tra định dạng code, lỗi cú pháp.

Tự động hóa build và kiểm thử:

- Build Docker image cho API Node.js.
- (Tùy chọn) chạy unit test bằng Jest nếu có test script.

Tự động triển khai:

- Khi code được push lên main, hệ thống sẽ tự động deploy container mới.

4.3. Cấu hình Docker và quy trình triển khai

Nhằm đảm bảo tính đồng nhất giữa các môi trường, nhóm sử dụng Docker để container hóa toàn bộ hệ thống. Quy trình bao gồm:

Viết Dockerfile:

- Khai báo môi trường Node.js.
- Cài đặt các gói phụ thuộc.
- Mở port ứng dụng và khởi chạy server.

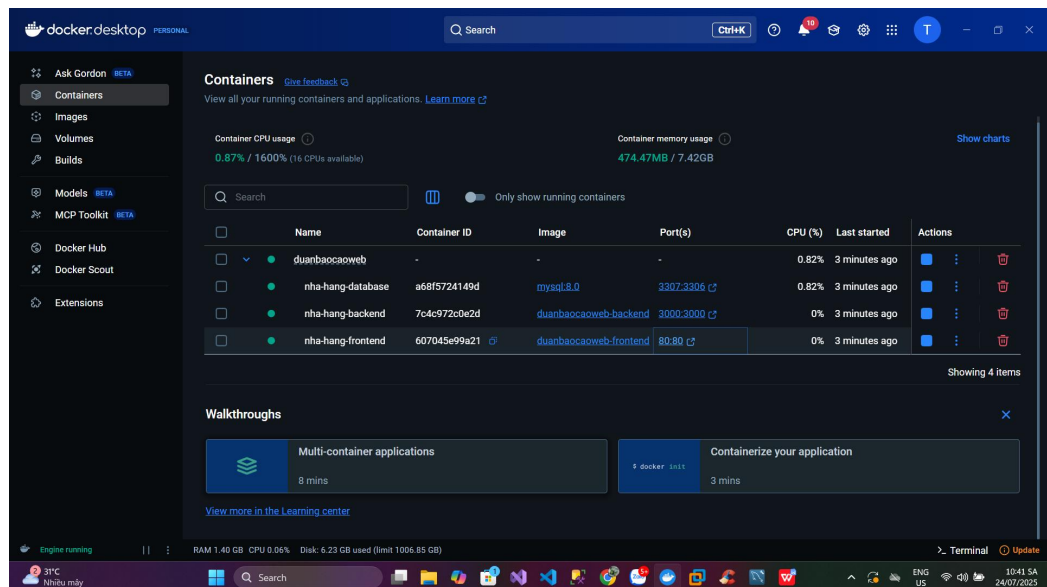
Cấu hình docker-compose.yml:

- Gồm hai services: app (API server) và db (MySQL)
- Tự động tạo volume lưu trữ và kết nối giữa các container.

Các bước triển khai thực tế:

- `docker build -t amthucpn`
- `docker-compose up -d`
- Đưa image lên Docker Hub hoặc deploy trực tiếp bằng Railway/VPS.

4.4. Kết quả kiểm thử docker



Hình 4.2: Kết quả kiểm thử Docker (Frontend, Backend, DataBase)

CHƯƠNG 5. QUẢN LÝ DỰ ÁN

5.1. Vận dụng mô hình phát triển phần mềm Agile

Sprint	Tuần	Mục tiêu Sprint	Nội dung chính
Sprint 1	Tuần 1	Làm quen Node.js & môi trường phát triển	- Cài đặt Node.js, npm- Tạo server đầu tiên- In “Hello Server” ra trình duyệt
Sprint 2	Tuần 2	Làm chủ Express – Xây dựng routing & middleware	- Cài đặt Express- Tạo routes GET/POST- Middleware xử lý request, response
Sprint 3	Tuần 3	Thiết kế cấu trúc dự án & tổ chức code MVC	- Phân tách: routes / controllers / models- Cấu trúc rõ ràng, dễ bảo trì
Sprint 4	Tuần 4	Kết nối cơ sở dữ liệu & tạo bảng món ăn / khách hàng	- Thiết lập MySQL- Tạo bảng món ăn, khách, đặt bàn- Kết nối DB từ Node.js
Sprint 5	Tuần 5	Xây dựng API CRUD món ăn (menu)	-TạoAPI GET/POST/PUT/DELETE- Xử lý dữ liệu món ăn- Kiểm tra với Postman
Sprint 6	Tuần 6	Xây dựng API CRUD đặt bàn & thông tin khách hàng	- Tạo API đặt bàn- Validate input- Lưu thông tin khách

TÌM HIỂU VỀ NODEJS VÀ XÂY DỰNG WEBSITE NHÀ HÀNG ẨM THỰC PHƯƠNG NAM TRÀ VINH

Sprint	Tuần	Mục tiêu Sprint	Nội dung chính
			hàng
Sprint 7	Tuần 7	Thiết kế giao diện tĩnh: menu, đặt bàn, giới thiệu	- Sử dụng HTML/CSS hoặc Tailwind- Trang chủ, trang menu, form đặt bàn
Sprint 8	Tuần 8	Kết nối front-end ↔ back-end (API)	- Gọi API hiển thị danh sách món- Submit form đặt bàn- Thông báo sau khi đặt thành công
Sprint 9	Tuần 9	Tối ưu trải nghiệm người dùng (UI/UX) + responsive	- Responsive cho mobile/PC- Căn lề, màu sắc, font chữ- Hover, trạng thái, hiệu ứng nhẹ nhàng
Sprint 10	Tuần 10	Kiểm thử hệ thống, sửa lỗi & triển khai lên cloud/docker	- Test toàn bộ API- Sửa bug- Viết Dockerfile- Deploy, tạo README

CHƯƠNG 6. KIỂM THỬ

6.1. Chiến lược kiểm thử

Dự án sử dụng kết hợp kiểm thử thủ công bằng Postman và kiểm thử tự động bằng GitHub Actions (nếu có test script).

Kiểm thử chức năng (Functional Testing):

- Gửi request tới các API: đặt bàn, xem món ăn, CRUD món, kiểm tra phản hồi.

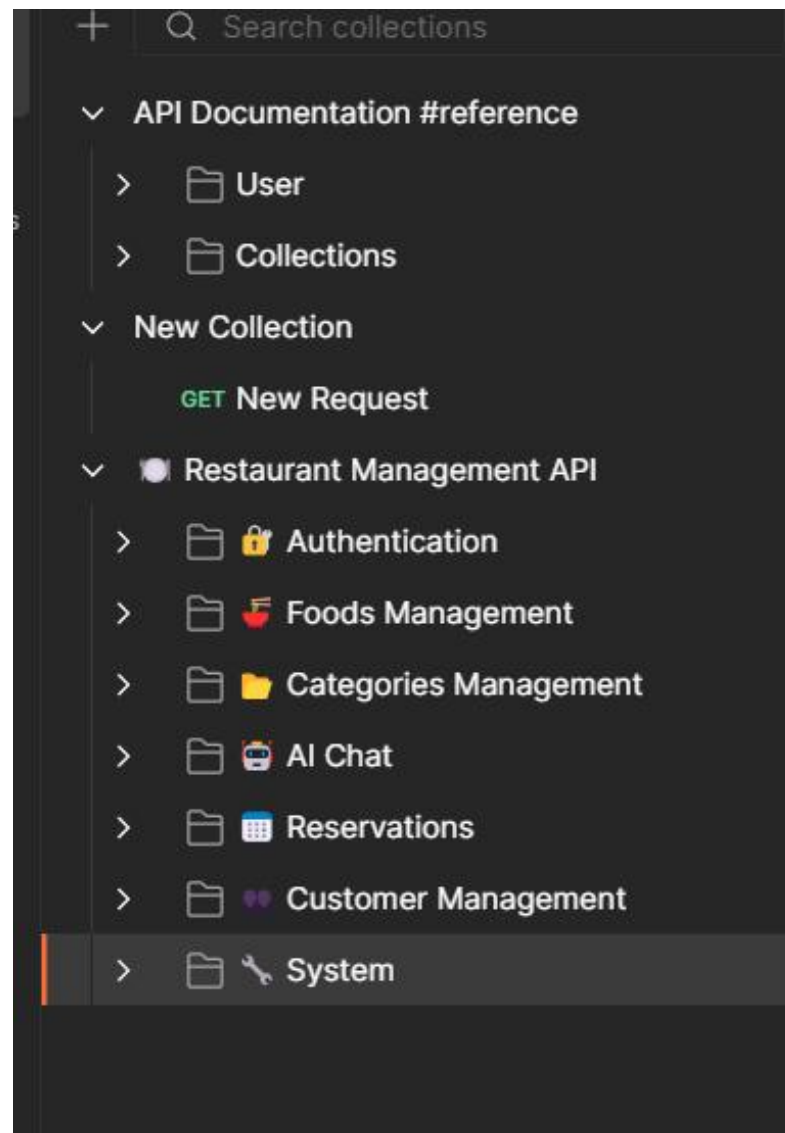
Kiểm thử API bằng Postman:

- Tạo Collection đầy đủ cho các chức năng.
- Dùng Postman runner để chạy tất cả endpoint.

Kiểm thử CI/CD:

- Mỗi lần push code, GitHub Actions tự động build → kiểm thử (test script) → kiểm tra lỗi.

6.2. Kết quả kiểm thử API



Hình 6.1: Kiểm thử các API

CHƯƠNG 7. ĐÁNH GIÁ VÀ KẾT LUẬN

7.1. Những khó khăn gặp phải

- Ban đầu chưa quen với Node.js và Express dẫn đến khó khăn khi thiết kế API.
- Quản lý database phức tạp khi chia nhiều bảng liên kết.
- Khó khăn khi triển khai Docker do chưa quen lệnh và cấu hình.
- Làm việc nhóm gặp trở ngại khi thời gian biểu mỗi thành viên không đồng bộ.

7.2. Bài học rút ra

- Hiểu sâu hơn về mô hình MVC, cách tổ chức dự án backend hiện đại.
- Nắm được quy trình triển khai phần mềm theo mô hình Agile và CI/CD thực tế.
- Rèn luyện kỹ năng làm việc nhóm và giao tiếp công nghệ qua Jira, GitHub.
- Học được cách đưa dự án vào container, giúp sản phẩm dễ triển khai hơn.

7.3. Đề xuất cải tiến trong tương lai

- Tách hệ thống thành các microservices để dễ mở rộng quy mô.
- Thêm tính năng xác thực người dùng bằng JWT.
- Tích hợp cổng thanh toán (VNPay, Momo).
- Phát triển ứng dụng mobile Flutter kết nối cùng API hiện tại.

CHƯƠNG 8. PHỤ LỤC

8.1. Hướng dẫn cài đặt và chạy ứng dụng

A. Điều kiện môi trường

- Hệ điều hành: Windows / macOS / Linux
- Cài đặt trước:
- Docker Desktop
- Git
- Node.js (nếu chạy không dùng Docker)

B. Cài đặt và chạy bằng Docker (Khuyến khích)

Clone dự án từ GitHub:

```
git clone https://github.com/KyThuatTVU/DuAn-WebSiteNhaHang.gitcd DuAn-WebSiteNhaHang
```

Khởi chạy Docker container:

```
docker-compose up -d
```

Truy cập ứng dụng tại:

```
http://localhost:3000
```

DANH MỤC TÀI LIỆU THAM KHẢO

Node.js Documentation. (n.d.). Node.js official docs. Truy cập tại:
<https://nodejs.org/en/docs>

Express.js Guide. (n.d.). Express.js documentation. Truy cập tại:
<https://expressjs.com/>

MySQL Documentation. (n.d.). MySQL Reference Manual. Truy cập tại:
<https://dev.mysql.com/doc/>

Docker Docs. (n.d.). Docker Documentation. Truy cập tại:
<https://docs.docker.com/>

GitHub Actions. (n.d.). Automate your workflow with GitHub Actions. Truy cập tại: <https://docs.github.com/en/actions>

Jira Software Guide. (n.d.). Atlassian Jira Documentation. Truy cập tại:
<https://support.atlassian.com/jira-software-cloud/>

Tailwind CSS Docs. (n.d.). Tailwind CSS Utility-first framework. Truy cập tại: <https://tailwindcss.com/docs>

Postman Learning Center. (n.d.). API testing with Postman. Truy cập tại:
<https://learning.postman.com/>

Railway Deployment Docs. (n.d.). Deploy Node.js app to Railway. Truy cập tại: <https://docs.railway.app/>

YouTube – F8 Official. (2023). Lập trình Node.js cơ bản & REST API. Truy cập tại: <https://www.youtube.com/c/F8VNOOfficial>

W3Schools. (n.d.). HTML, CSS, and JavaScript tutorials. Truy cập tại:
<https://www.w3schools.com/>